

MATLAB foundation -2

Determining Size

- The *size* function returns the number of rows and columns as either a vector or two separate scalars:

```
>> s = size(Prices)  
>> [m, n] = size(Prices)
```
- The *length* function returns the largest value returned by size:

```
>> length(Prices)
```
- The *numel* function returns the total number of elements:

```
>> numel(Prices)
```

Matrix Operations

- As with vectors, arithmetic operations on matrices are defined in their usual mathematical sense.

```
>> C = A + B;
```

— creates a matrix C such that $C(k)$ is $A(k) + B(k)$.

```
>> C = k * A;
```

```
>> C = A + k;
```

- Mathematically, matrix multiplication of the form

```
>> C = A * B
```

requires that the inner dimensions --- the number of columns in A and the number of rows in B --- agree. If A is m-by-n and B is n-by-p, then C will be m-by-p.

Array Operations

- Can perform an array. Simply prefix the operator with a dot (.):
 - >> GMRatio = Germany ./ Mexico
 - Simply, to calculate the geometric mean of the German and Mexican prices, enter
 - >> GMProd = Germany .* Mexico
 - >> GMGMean = *sqrt*(GMProd)

Mathematical Operations

Performed on elements of a matrix

$$\sin \begin{pmatrix} a & b \\ c & d \end{pmatrix} = \begin{pmatrix} \sin(a) & \sin(b) \\ \sin(c) & \sin(d) \end{pmatrix}$$

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \rightarrow \boxed{\sin} \rightarrow \begin{pmatrix} 0.8415 & 0.9093 \\ 0.1411 & -0.7568 \end{pmatrix}$$

`sin`

`sin d`

`sinh`

`a sin`

`exp`

`log`

`log 2`

`log10`

`sqrt`

`nthroot`

`abs`

`angle`

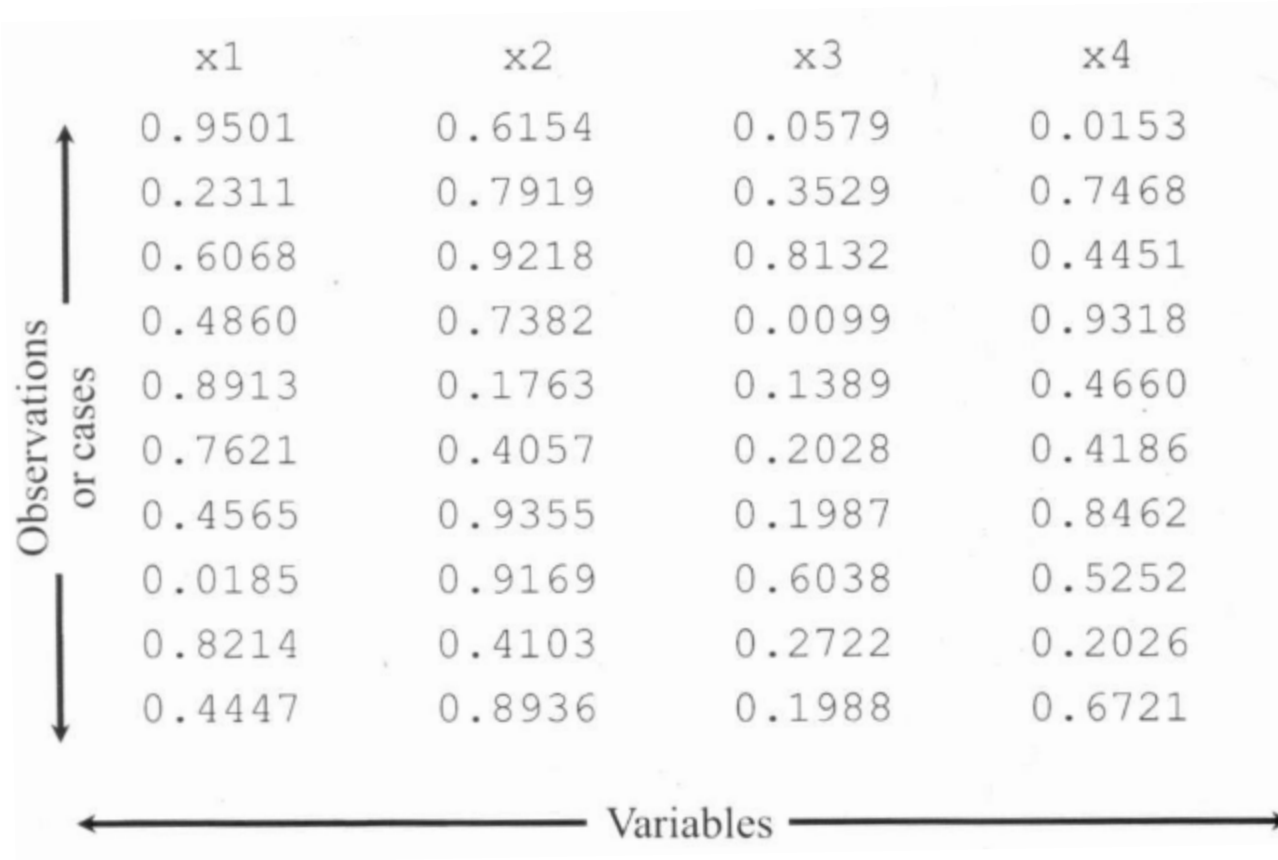
`floor`

`ceil`

`round`

`mod`

Data in the MATLAB® Environment



The diagram shows a data matrix with 10 rows and 4 columns. The vertical axis is labeled 'Observations or cases' with an upward arrow, and the horizontal axis is labeled 'Variables' with a rightward arrow. The columns are labeled x1, x2, x3, and x4. The rows contain numerical values for each variable.

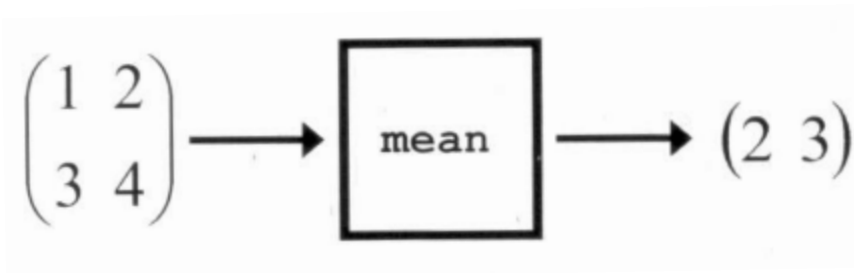
	x1	x2	x3	x4
↑	0.9501	0.6154	0.0579	0.0153
	0.2311	0.7919	0.3529	0.7468
	0.6068	0.9218	0.8132	0.4451
	0.4860	0.7382	0.0099	0.9318
	0.8913	0.1763	0.1389	0.4660
	0.7621	0.4057	0.2028	0.4186
	0.4565	0.9355	0.1987	0.8462
	0.0185	0.9169	0.6038	0.5252
	0.8214	0.4103	0.2722	0.2026
↓	0.4447	0.8936	0.1988	0.6721

Statistical functions treat the columns of a matrix independently

Statistical Operations

Performed on an entire vector
or on the columns of a matrix

$$\begin{array}{cc} \text{mean} \left(\begin{array}{cc} a & b \\ c & d \end{array} \right) = \left(\text{mean}(v_1) \text{mean}(v_2) \right) \\ \uparrow \quad \uparrow \\ v_1 \quad v_2 \end{array}$$



max

min

mean

median

std

sum

prod

diff

gradient

cumsum

cumprod

cov

filter

conv

fft

ifft

Reshaping

A =

1	2	3
4	5	6
7	8	9
0	1	2

```
>> B=reshape(A, 2, 6)
```

B =

1	7	2	8	3	9
4	0	5	1	6	2

```
>> C=reshape(A, 6, 2)
```

C =

1	8
4	1
7	3
0	6
2	9
5	2

Linear Indexing

```
>> A=magic(5)
```

A =

17	24	1	8	15
23	5	7	14	16
4	6	13	20	22
10	12	19	21	3
11	18	25	2	9

A(1)=17, A(2)=23, A(3)=4, A(4)=10, A(5)=11,

A(6)=24, ...

...

A(25)=9

Multidimensional Arrays

```
>> B=rand(2,2,3)
```

```
B(:, :, 1) =
```

0.0605	0.5269
0.3993	0.4168

```
B(:, :, 2) =
```

0.6569	0.2920
0.6280	0.4317

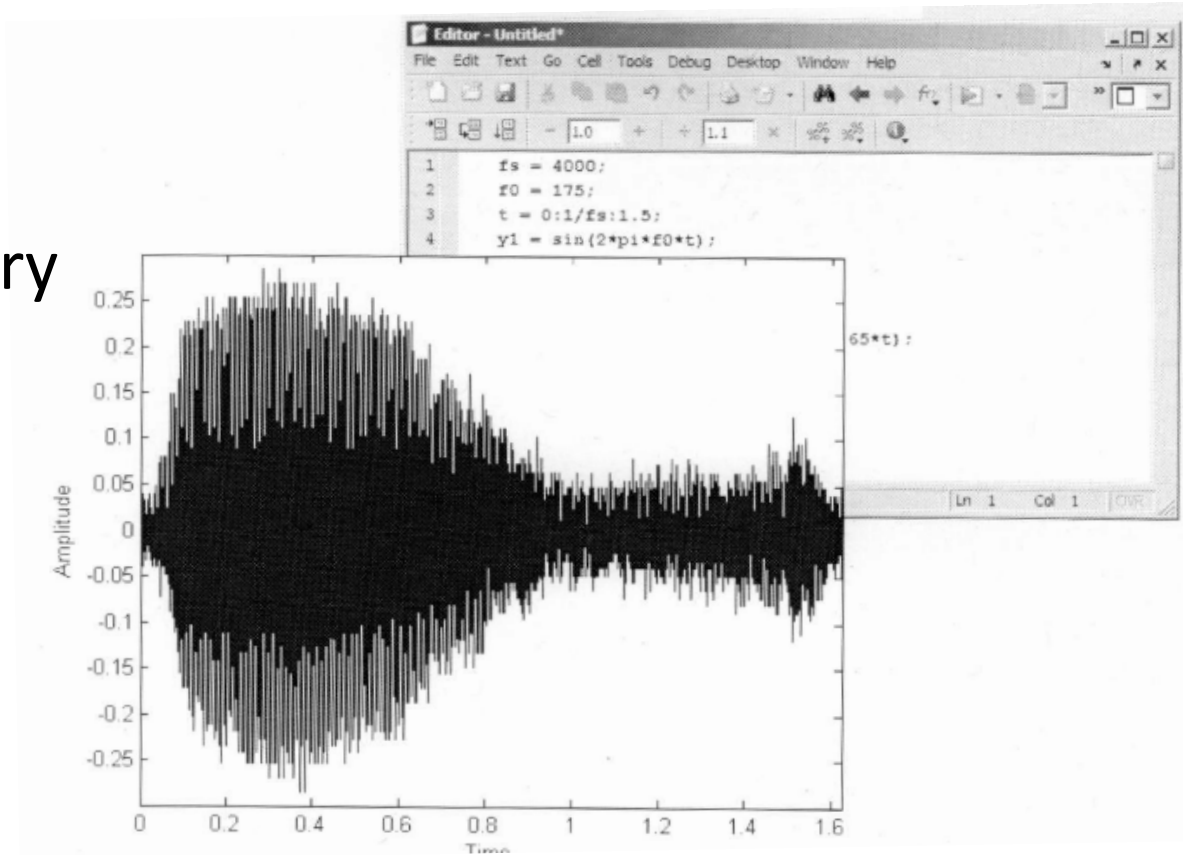
```
B(:, :, 3) =
```

0.0155	0.1672
0.9841	0.1062

四、Automating Commands with Scripts

Outline

- A modeling example
- The Command History
- Crating script files
- Running scripts



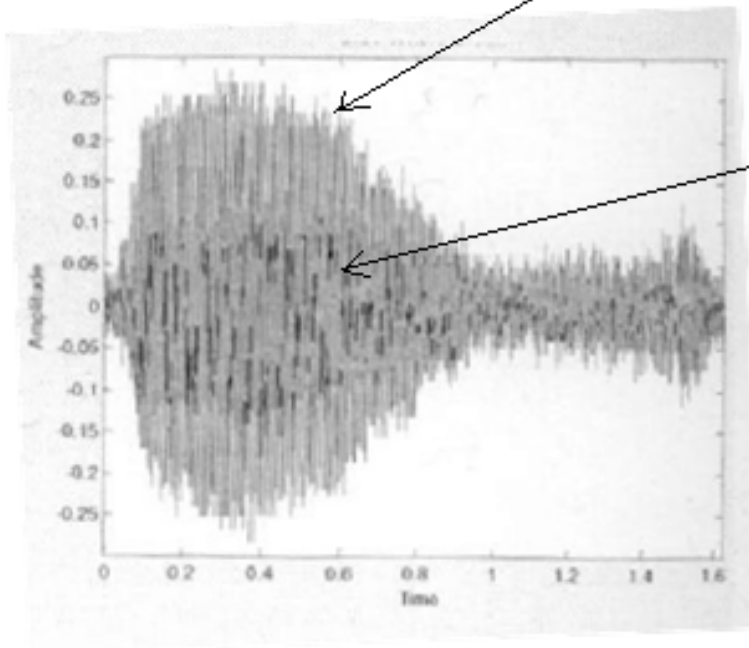
Modeling a Whale Call

Amplitude modulated $y(t) = A(t)y_0(t)$



Decaying oscillation

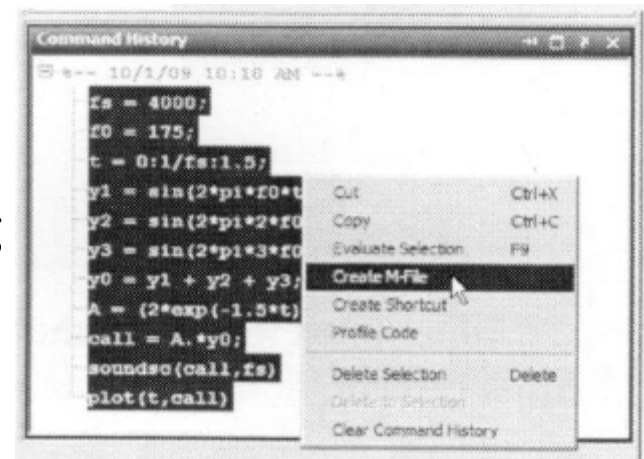
Sum of harmonics of a
a fundamental frequency



$$y_0(t) = \sum_n \sin(2\pi n f_0 t)$$

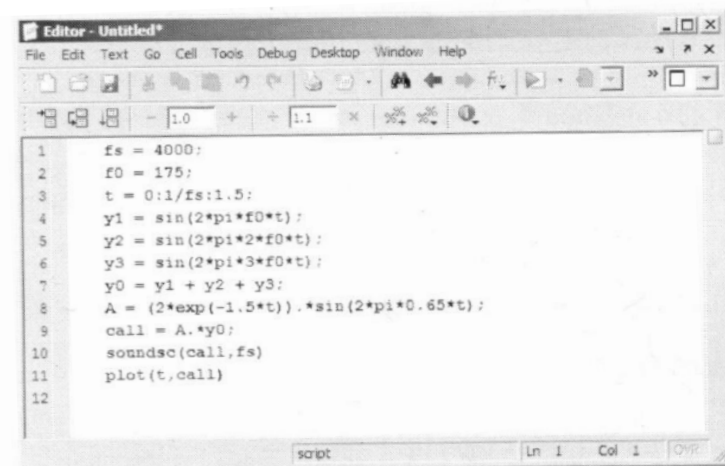
The Command History

- MATLAB keeps a history of the commands you type, and displays them in the **Command History window** on the MATLAB desktop.
- These commands remain from session to session until you choose to delete them, and they can be referenced from the command prompt using the **up arrow key** ↑ or by dragging (click and holding) them from the command History and dropping them at the prompt.
- You can select a range of commands in the history by holding down the **Shift** key, or separated commands by holding down the **Ctrl** key.



The MATLAB® Editor

- choose “**Create script**” from the command history by right-click
- choose “**HOME-New-Script**” from the menus at the top of the MATLAB desktop
- `>> edit` can open the editor and begin a new file
- `>> edit filename` can open an existing file in the editor.



Script Files

- Scripts do **not have inputs and outputs**. MATLAB executes the commands as a batch, in order.
- Scripts are used to automate useful blocks of commands.
- Text following a percent sign(**%**) **is a comment**.
- The first continuous block of comments in a file is reserved for help information. Help is displayed when you use the syntax
`>> help filename`
- The first comment line in the help is called the *H1 line*. It should contain the filename and a brief description of the purpose of the file.

Running a Script

- Run a script by typing the file name (without the .m extension) if it is in the current directory.

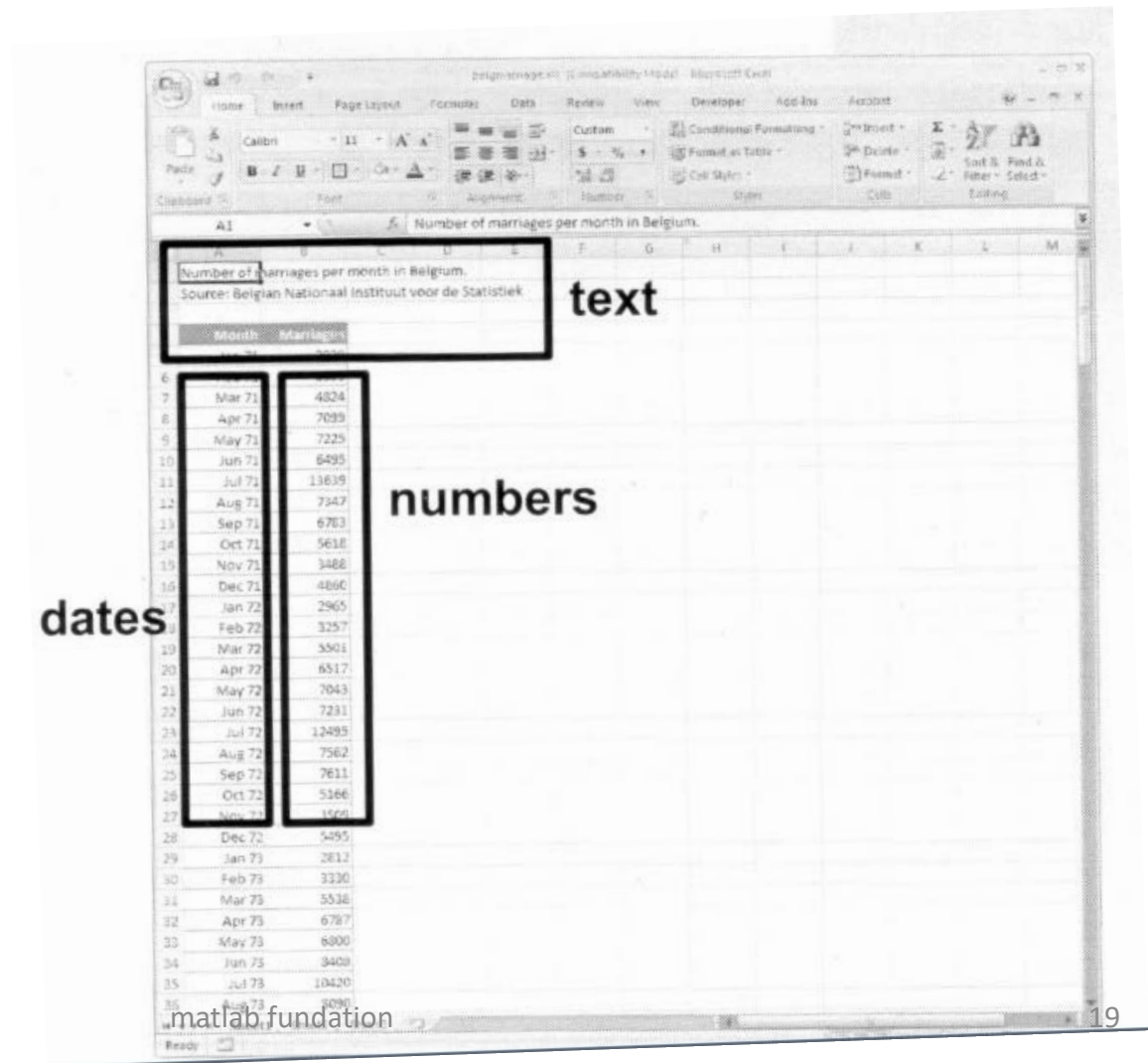
```
>> callmodel
```

- Click on the Run button at the top of the Editor to run a script if it is displayed in the Editor. Equivalently, you can press **F5**.

五、Working with Data Files

Outline

- Importing data
- Cell arrays
- Numbers, strings, and dates
- Exporting data

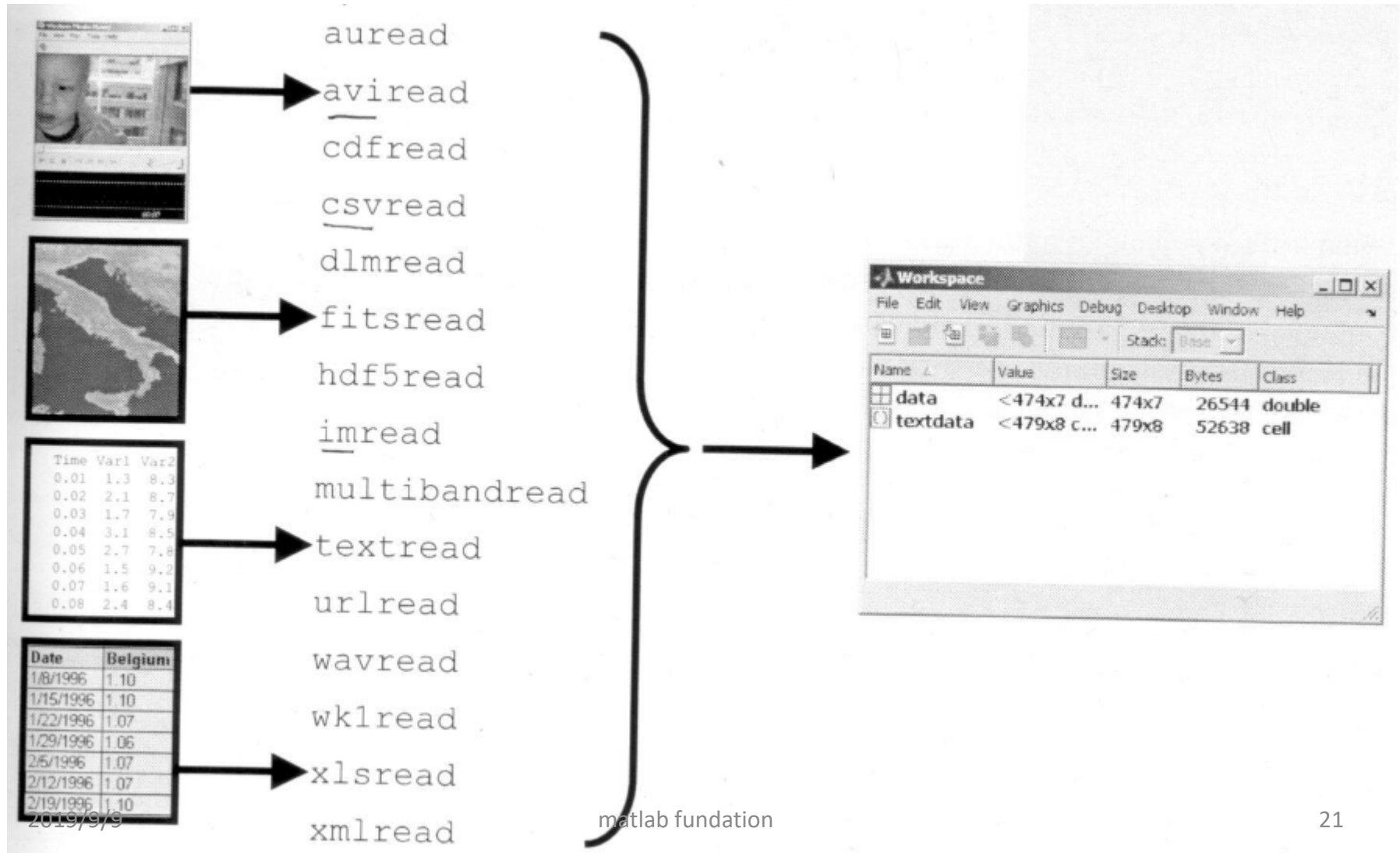


File Types and File Formats

- File type refers very generally to the origins of the data
- File format refers the encoded in a file

File Type	File Format	Import Wizard
Audio		
Computer Aided Design (CAD)		
Document		
Spreadsheet		
Database	WAV	
Presentation	AIFF	wavread
Webpage	RAW	
Script	AU	
Page description	MP3	
Graphics	RA	
Video	MIDI	
Scientific data	...	
Geographic information		

The *read Functions



Mixed Data Types

- Often data files contain a mix of data types, such as text and numeric values. For example,

```
>> M = xlsread('belgmarriage.xls')
```

- read in only the numeric data.

```
>> [M, txt] = xlsread('belgmarriage.xls')
```

- To avoid this common problem, many import functions in MATLAB return the data in *cell arrays*.

Cell Arrays

- MATLAB arrays must always have the same number of columns in each row(and vice versa)
- A single string is treated as a 1-by-n array of characters.
`>> x = ['hello'; 'ciao'; 'guten tag']----- error`
 - Vertical concatenation of strings fails unless they are all the same length.
- To concatenate elements into a [cell array](#), use curly braces (`{ }`) instead of square brackets:
`>> x = {'hello'; 'ciao'; 'guten tag'}-----correct, 3*1,each container has 1*n.`
`>> y = {pi, []; 'foo', 42; ones(2), eye(3)}`
`>> z = {x, y}`
`>> z = {x; y}`
`>> z = [x, y] % ok`
`>> z = [x; y] % no! dimension mismatch`

Cell Indexing

- Indexing into cell arrays is very similar to indexing into any other type of array.
- MATLAB provides two ways to index into cell arrays:
 - use parentheses to reference the container
`>> a = x(1) % a is a 1-by-1 cell array (the container),`
 - use curly braces to reference the contents:
`>> b = x{1} % b is a 1-by-5 character array (the contents).`

Converting Numbers and Strings

- *num2str*
- *str2double*, *str2num*

```
>> num2str(42)
>> num2str(pi)
>> str2num('42')      % for scalars
>> str2double('42')   % for scalars...
>> str2double({'42';'3.14'}) % ...or arrays
```

You can also use display functions such as `sprintf` and `fprintf` to produce strings of numbers in a specific format:

```
>> x = sprintf('pi is equal to %7.5f...',pi)
```

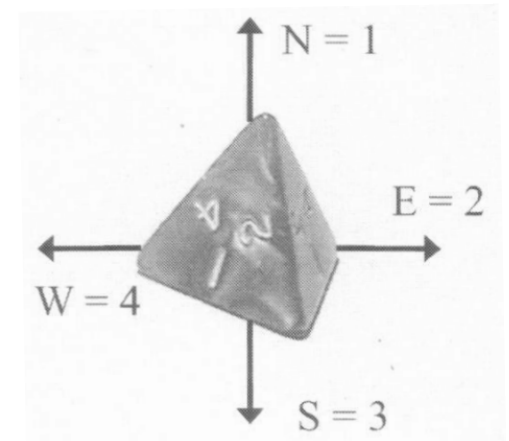
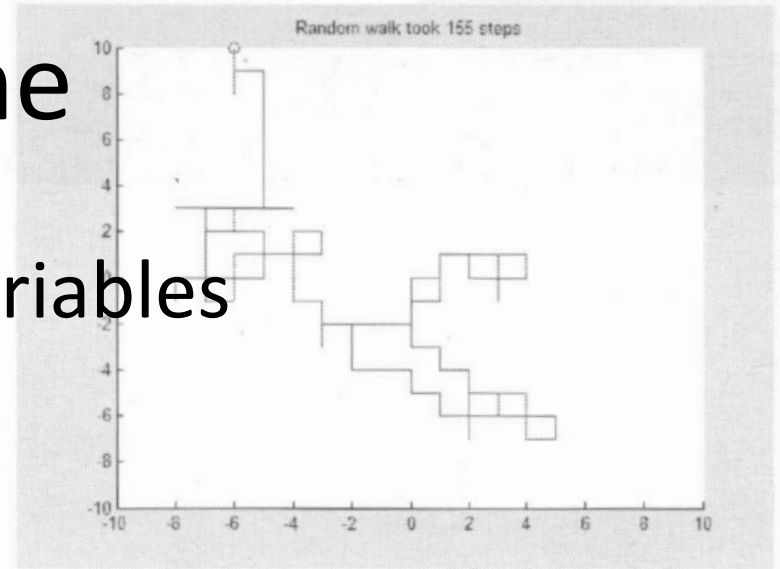
For a list of conversion functions, enter

```
>> help strfun
```

六、 Logic and Flow Control

Outline

- Logical operations and variables
- Logical indexing
- Flow control
- Loops
- User input and output



Logical Operations and Variables

- logical variables have only two possible values: true and false, 1 and 0

```
>> x = pi > 3 %x=1, which is a logical variable
```

- To test for equality, use a double equal sign, which is the logical equality operator in MATLAB:

```
>> x = pi == 3
```

```
>> x = (pi > 3) & (pi < 4)
```

- all the relational and logical operators

```
>> help ops
```

Logical Functions

Operator	Operation
true, false	Setting logical value
& (and), (or), ~ (not), xor, any, all	Logical operations
&&,	Short-circuit operations
bitand, bitcmp, bitor, bitmax, bitxor, bitset, bitget, bitshift	Bitwise operations
== (eq), ~= (ne), < (lt), > (gt), <= (le), >= (ge)	Relational operations
All is* functions	Test operations
strcmp, strncmp, strcmpi, strncmpi	String comparisons

Finding and Counting

- The *find* function returns the indices of true values in a logical array.

```
>> find(HiMex)
```

```
>> find(Mexico > 2.2)
```

- One way to count the number of trues in a logical array is to apply the *sum* function:

```
>> sum(HiMex)
```

- An alternative solution is provided by the *nnz* function, which returns the number of nonzero elements in an array:

```
>> nnz(HiMex)
```

Logical Indexing

- MATLAB provides an elegant way to index into arrays without having to use *find*: *logical indexing*.
- You can use a logical variable as an array index, in which case MATLAB extracts the array elements where the index is *true*:

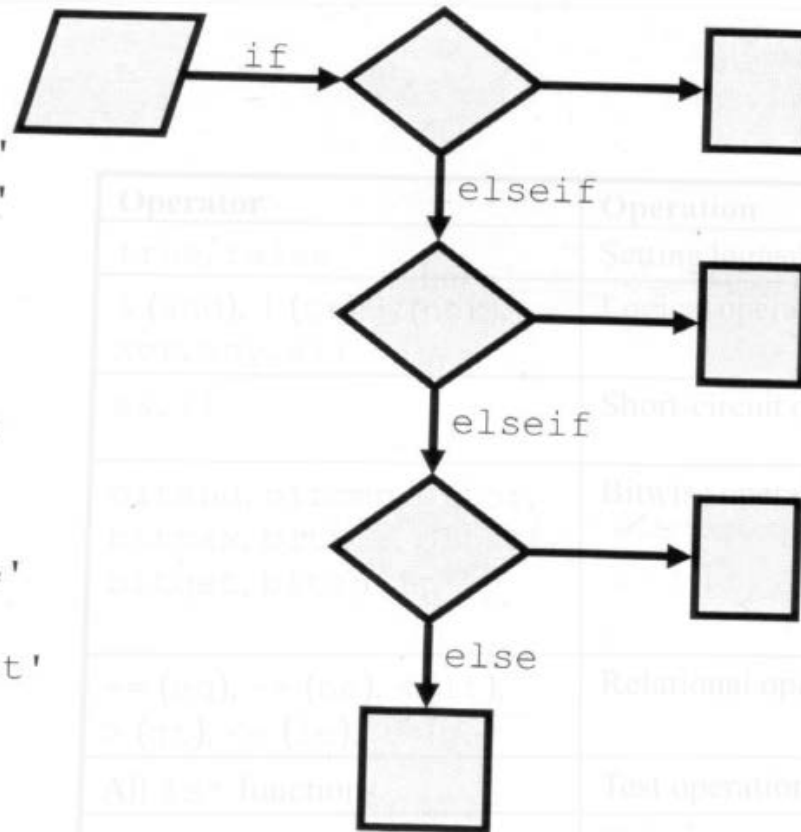
```
>> x = [42, pi, 0, 7];  
>> y = [true, false, false, true];  
>> x(y)
```

- This is commonly done by using a logical condition to create the logical index:

```
>> y = (x > 4)  
>> x = x(y)  
or even just  
>> x = x(x > 4)
```

Programming Keywords

```
>> iskeyword
ans =
'break'
'case'
'catch'
'classdef'
'continue'
'else'
'elseif'
'end'
'for'
'function'
'global'
'if'
'otherwise'
'parfor'
'persistent'
'return'
'spmd'
'switch'
'try'
'while'
```



Flow Control

- Flow control constructions allows you to introduce conditional branching into your programs.

```
if (logical test 1)  
    statements 1  
elseif (logical test 2)  
    statements 2  
else  
    statements 3  
end
```

```
switch expression  
    case value 1  
        statements 1  
    case value 2  
        statements 2  
    otherwise  
        statements 3  
end
```

example

- Example for 'if'
 - 'ifelse.m'
- Example for 'switch'
 - 'switchcase.m'

For-Loops

```
for index = first:increment:last  
    statements  
end
```

Example:

```
v = [1,2,3,5,8,13];  
for k = v  
    statements  
end
```

Another example 'forloop.m'

While-Loops

```
while condition  
    statements  
end
```

Example:

```
x=3;  
while (x>2)  
    x=2*x;  
end
```

Another example 'whileloop.m'

Providing Output

- To display information while the program is running, the MATLAB commands *disp*, *fprintf*, and *sprintf* are useful.
 >> *disp('Ciao')*
 >> *fprintf('%s\n', 'Ciao')*
 >> *disp(sprintf('%s\n', 'Ciao'))*
- It can also provide *errors* and warnings to users using error and warning.

```
if (x<0)  
    error('Negative square root?! I quit!')  
else  
    y=sqrt(x);  
end
```

Obtaining User Input

- The *input* command pauses execution and displays a message (or prompt) to the user.

uage = input('Please enter your age: ');

- When expecting text input, add 's' option to the input function.

uname = input('Please enter your name: ', 's');

七、函数编写与调试

Matlab的两种源程序文件格式

- M-脚本文件（M-script）
 - 包含一组由matlab语言所支持的语句。
 - 执行：在提示符下输入该文件名
 - M-script 适合于小规模运算
- M-函数格式
 - 每个M-函数文件包括一个主函数
 - 函数的调用通过M-函数文件名，再加参数调用。
 - M-函数是 MATLAB 编程的主流方法

【例】求满足下列求和公式的最小的 m 值；

$$\sum_{i=1}^m i > 10000$$

```
>> s=0;m=0;  
>> while (s<=10000),m=m+1;s=s+m;end,[s,m]  
  
ans =  
  
    10011         141
```

若最大值不为 10000，需修改程序
对 m 和 10000 值的设置，不适合于M-script

MATLAB 语言函数的 基本结构

Function[返回变量列表]=函数名（输入变量列表）
注释说明语句段，由%引导
输入、返回变量格式的检测
函数体语句

- **nargin, nargout:** Matlab保留变量，只要进入函数就自动生成这2个变量，分别表示输入、输出参数的个数
- **varargin:** 所有输入变量列表；
- **varargout:** 所有输出变量列表；

【例】前面的要求， m , 10000

```
function [m,s]=findsum(k)
s=0;m=0;
While(s<=k), m=m+1;s=s+m;end
```

```
>> [m1,s1]=findsum(145323)
```

- 无需修改程序
- 见文件findsum.m

【例】编写一个函数生成 $n \times m$ 阶的Hilbert矩阵

Hilbert矩阵:

第 i 行第 j 列的元素值为: $h_{i,j} = 1/(i + j - 1)$

- 若只给出一个输入参数，则会自动生成一个方阵
- 在函数中给出合适的帮助信息
- 检测输入和返回变量的个数

% Look the file '**myhilbfun.m**'

```
function A = myhilb(n, m)  
if nargout>1, error('too many output arguments.');  
end  
if nargin==1,m=n;  
elseif nargin==0|nargin>2  
    error('wrong number of input arguments.'); end  
for i=1:n  
    for j=1:m  
        A(i,j)=1/(i+j-1);  
    end end  
end
```

```
>> help myhilbfun
```

```
>> myhilbfun(3,4)
```

```
>> myhilbfun(3)
```

【例】函数的递归调用：阶乘

```
function k=my_fact(n)
if nargin~=1,error('Error of the number of input'); end
if nargout>1,error('Error of the number of output');end
if abs(n-floor(n))>eps | n<0,
    error('n is not a positive integer'; end
if n>1      %若n>1，进行递归调用
    k=n*my_fact(n-1);
elseif any([0 1]==n)      %0!=1!=1已知
    k=1;
end
```

```
>>my_fact(1)
```

```
>>my_fact(11)
```

```
>>my_fact(170)
```

```
>>my_fact(300)
```

【例】 比较递归算法和循环算法在Fibonacci数列中应用的优劣

Fibonacci数列: $a_1=a_2=a$, 第k项 ($k=3, 4, 5\cdots$)
可以写成 $a_k=a_{k-1}+a_{k-2}$

- 递归算法

```
function a=my_fibo(k)
```

```
if k==1 |k==2, a=1;
```

```
else, a=my_fibo(k-1) +my_fibo(k-2); end
```

- 循环算法: `my_fibo_for.m`

```
function a = my_fibo_for( k )
```

```
a=[1,1];
```

```
for(t=3:k)
```

```
    a(t)=a(t-1)+a(t-2);
```

```
end
```

```
end
```

比较两种算法:

`tic,...,toc`: 计算在这两个之间的计算所花费的时间

```
>>tic, my_fibo(25); toc
```

```
>>tic, my_fibo_for(25); toc
```

可变输入输出个数

- `nargin`, `nargout`: 分别表示输入、输出参数的个数
- `varargin`: 所有输入变量列表;
- `varargout`: 所有输出变量列表;

可变输入输出个数

【例】计算多个多项式的积（convs.m文件）

提示：conv()，系统函数，计算两个多项式的积

```
function a=conv(varargin)
```

```
a=1;
```

```
for i=1: length(varargin),  
    a=conv(a,varargin{i});
```

```
end
```

```
>>P=[1 2 4 0 5]; Q=[1 2]; F=[1 2 3];  
D=convs(P,Q,F)
```

summary

- Working with the user interface
- Variables and expressions
- Analysis and visualization with vectors and matrices
- Automating commands with scripts
- Working with data files
- Logic and flow control
- function